Red Light, Green Light: Should He Go?

Abstract

The game of baseball is often decided by one or two runs, so base runners' decision to steal can have a high impact on winning or losing. This project uses SMT baseball data to study and analyze stolen base attempts from first base to second base. The data was filtered from ball-events, lineups, and play-positions then further narrowed down to plays involving a runner on first base, second base, short stop, and second baseman. Using Python and machine learning models, this project tested whether player movement features could help classify a steal attempt as successful or failed. The final model found that defensive movement features realted to defensive distance covered per step, velocity, acceleration, and jerk are the highest indicators for a steal prediction. Runner movement provided very high accuracy, so those results were concluded to be data leakage and not used in the final model. Throw-time analysis showed that catcher throws to the second baseman averaged 2.42 seconds with a 70% steal success rate, while throws to the shortstop averaged 2.41 seconds with a 62.75% steal success rate. Overall, the project found that the defensive movement helped predict stealing success.

Introduction

According to TeamRanking Sports Data, there were a total of 4,475 steals attempted during the 2025 MLB season. While this sounds like a large number, it averages only about .70 steals attempted per game. Because steals are a rare event, they create a valuable opportunity for fan engagement when they do occur. However, everyday fans may not fully understand the reasoning behind why a runner decided to steal. This project explores the opportuny of how can steal attempts be more exciting and easier for fans to understand.

The first goal of this project is to use machine learning to find patterns that can help predict when a base runner on first base can successfully steal second base. The second goal is to turn that prediction into a graphic design that fans can use to understand when a runner should steal. This project is titled "Red Light, Green Light: Should He Go because the model classifies a higher steal change a green light will be given, but a red light would mean to stay.

Data

The data for this project was provided by the SMT data challenge. The main data used were ball-events, lineups, and player-positioning. The player-positioning was filtered to runners on first and second base, second baseman, and short stop. A possible steal attempt was identified by looking at the change of bases between first and second. The logic was to search for plays where first base

was occupied, but second base was empty. Then the next play showed first base as empty and second base occupied. This suggests that the runner moved from first to second base, but this could also be a natural movement. To further filter the plays, lineups data was added to include the batter which changed the logic to if the batter stayed the same while the runner advanced from first to second then a steal happened.

Ball-event data was then added and used to calculate catcher throw time. The event sequence looked for a pitch, catcher catch, catcher throw, and then a catch by either the second base man or shortstop. Throw time was calculated by taking the difference between the catchers throw timestamp and the defensive catch timestamp

Analysis

The first thing that was compared was stealing outcomes from a defensive player that involved a throw from the catcher. In figure 1.1, the second baseman had 35 successful steals against him, with an average throw time of 2.42 seconds and a steal success rate of 70%. The shortstop had 32 successful steals against him, with an average throw time of 2.41 seconds and a steal success rate of 62.75%.

Red Light, Green Light

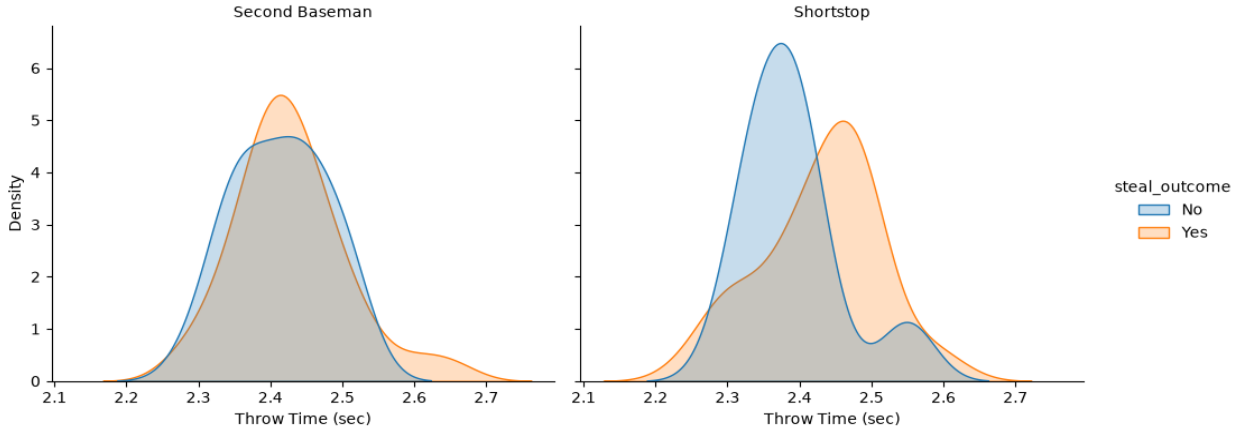
Figure 1.1

DEFENSIVE PLAYER	SUCCESSFUL STEALS	SUCCESS RATE	AVG THROW TIME
second baseman	35	70.00%	2.42
short stop	32	62.75%	2.41

This table compares steal outcomes when the catcher's throw was received by the second baseman or shortstop. Success rate shows the percentage of steal attempts that reached second base safely.

The next figure 1.2 shows the density plot for throw time. For throws involving the second base and, successful and failed steals had similar peaks between 2.4 and 2.5 seconds. For throws involving shortstop, failed steals peaked closer to 2.3 to 2.4, while successful steals peaked closer to 2.4 to 2.5 seconds. This suggests that the small differences in throw time can matter between a successful or failed steal.

Figure 1.2: Throw Time by Defensive Player and Steal Outcome



This figure compares catcher throw-time distributions for successful and failed steal attempts when the throw was received by the second baseman or shortstop.

Machine Learning

After isolating the steal attempts, additional movement columns were created. This included position, change in time, distance covered, velocity, acceleration, and jerk for both offensive and defensive players. The main features included in the model were: dcov_step_runnner/defense - defensive distance covered per step, i_vel_runner/defense - instantaneous velocity on how fast the player was moving, i_accel_runner/defense - instantaneous acceleration on how fast the players speed changed, i_jerk_runner/defense - instantaneous jerk on how suddenly the players acceleration changed.

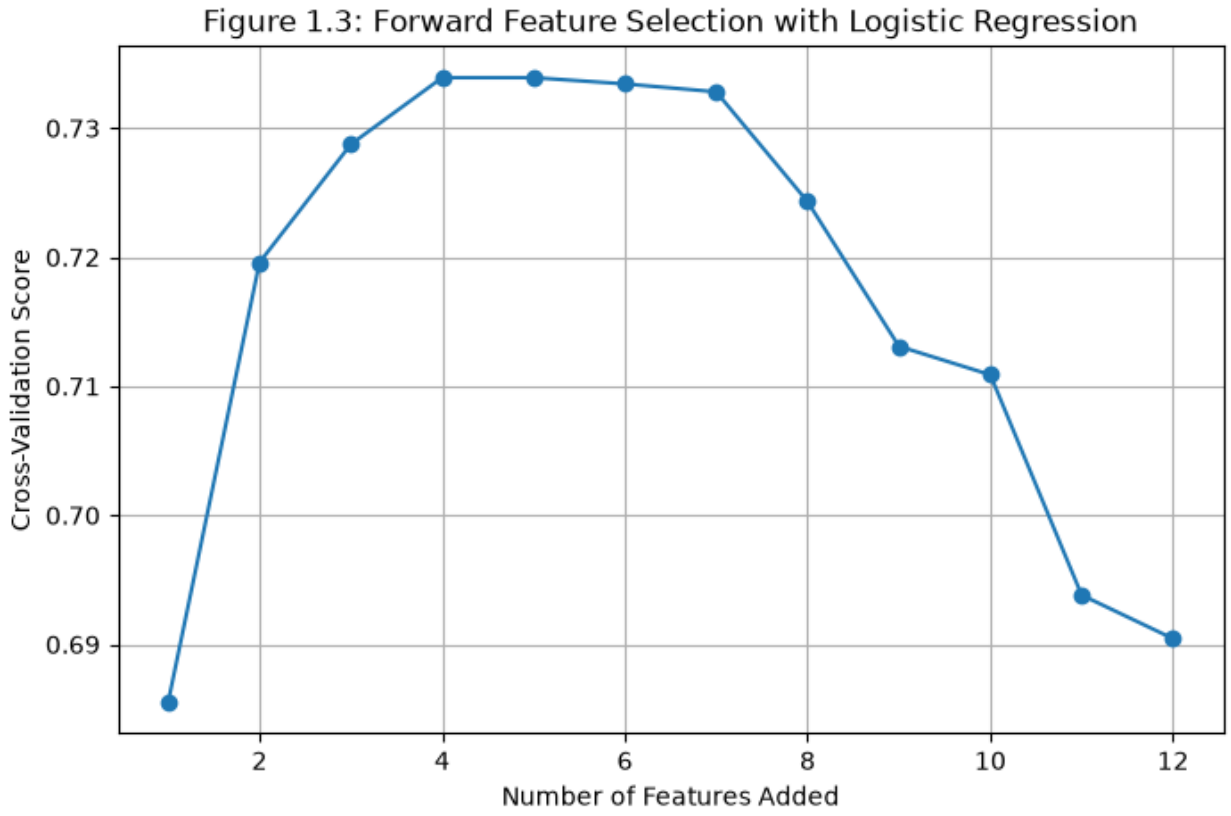
The main target variable for the mode was steal_successful, where 0 represents a failed steal and 1 is a successful steal. In machine learning language, the target variable is called y while the features used for predicting are called X.

Logistic regression was used for predicting the probability of a class such as successful or failed steals. Log regression is very similar to linear regression but is used for categorical outcomes. Decision trees were used for our model because they are easier to interpret and can be turned into a flow chart graphic. A decision tree works by splitting the data into small groups and assigning a predicted class to them.

Because the dataset had 24 possible features based on offensive or defensive positioning, testing every feature would be impossible with 24 features there are over 16 million possible combinations. Instead of manually testing every combination, forward feature selection was used. Forward feature selection starts with no features, then tests each feature one at a time. Then it keeps adding until it finds the best score.

Figure 1.3 shows the results plotted for forward feature selection using logistic regression. The model scores increased as the first few features were added. The highest score was

reached at four features than decreased every time a new feature was added. This shows that adding to many features is harmful and does not improve the model.



The model was scored using roc auc which measures how well the model separates failed and successful steals.

Model

Four models' tests were conducted, and the results are shown in figure 1.4.

Test 1 was conducted on all 24 features, and the model produced a score of 100%. This result was immediately rejected because 100% accuracy is highly unlikely. A perfect score suggests that the model is using information to reveal the answer instead of predicting it. This is called data leakage. After this result, a maximum score limit was set meaning that we automatically reject feature combinations that are higher than 90%.

Test 2 included all runner columns. Forward feature selection chooses six features with the highest score which includes runner position, throw time, runner accel, defensive change in time, and defensive distance covered per step. This model had a test score of 82% but was not included because it had similar results to test 3.

Test 3 removed all runner columns except runner positioning. Seven features were selected including runner positions, throw time, defensive change in time, defensive jerk, defensive y, and defensive distance covered per step. This models test score was the same as test 2 which was 82%. Because of this, the model was rejected.

Test 4 removed all columns that had any info on the runner. Four features were selected which include defensive velocity, defensive acceleration, defensive distance covered per step, and defensive jerk. This model has a test score of 74% and was chosen as the final model.

Model Testing

Figure 1.4

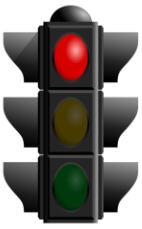
TEST #	# OF FEATURES	FEATURE LIST	TEST SCORE	NOTES
1	4	runner_y, i_vel_runner, runner_x, dcov_step_defense	100%	Removed max score and added all columns; likely data leakage
2	6	runner_y, runner_x, throw_time, i_accel_runner, change_time_defense_sec, dcov_step_defense	82%	Left in all runner columns
3	7	runner_y, runner_x, throw_time, change_time_defense_sec, i_jerk_defense, defense_y, dcov_step_defense	82%	Left in runner position columns
4	4	i_vel_defense, i_accel_defense, dcov_step_defense, i_jerk_defense	74%	Dropped all runner columns

Model 4 was chosen as the final model

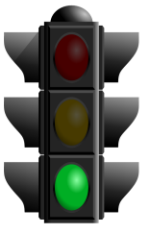
Conclusion

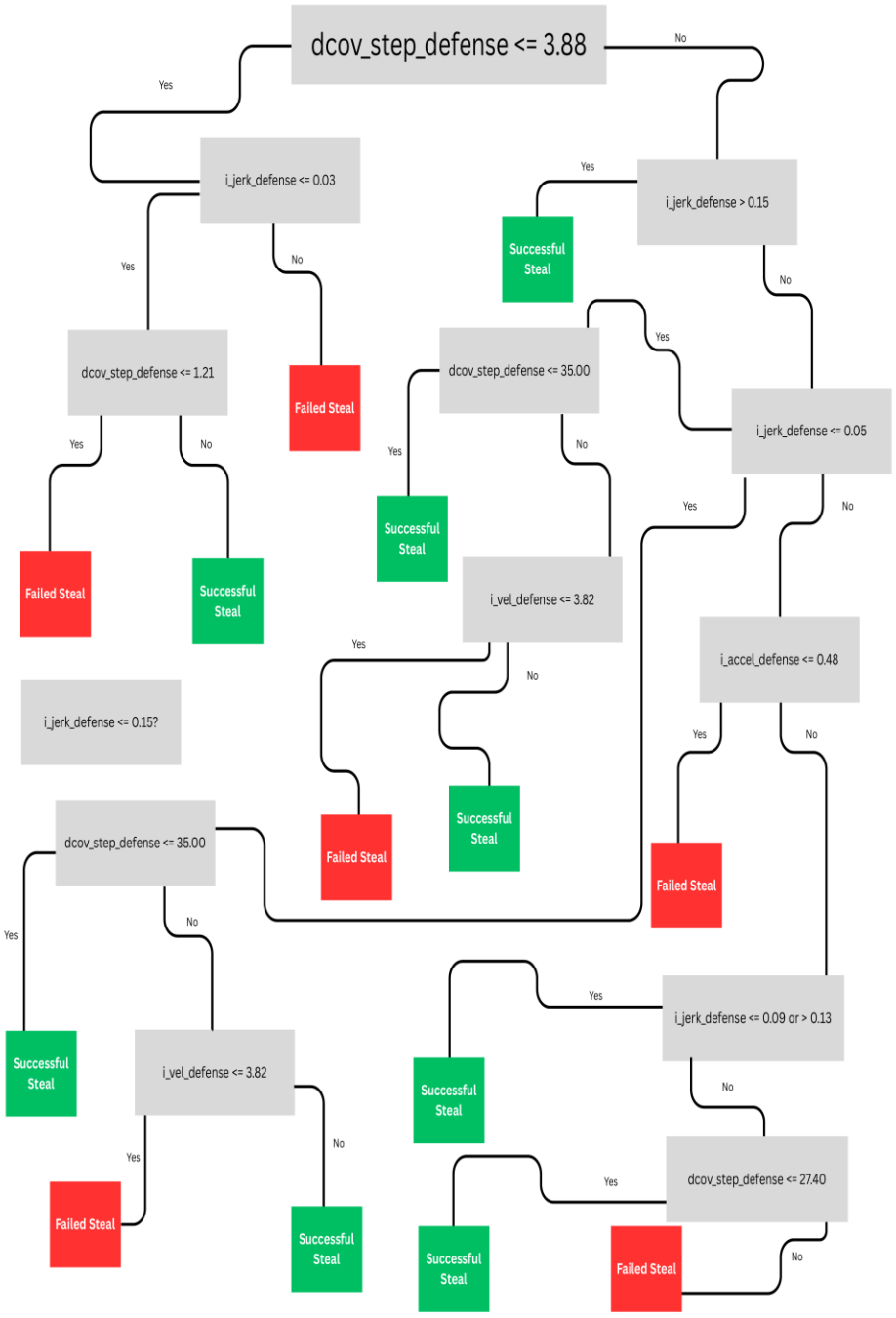
The final finding is that defensive movement patterns were important in predicting stealing success rates. Defensive distance covered per step, velocity, acceleration, and jerk helped the model classify whether a steal attempt was successful or failed. Ver small defensive movement was often related to a successful steal while larger more active defensive movement was connected to more failed steals. This does not mean that one feature guarantees a stealing outcome.

The fan-engagement idea is turned into a complex flow chart that people can use to get a better idea of whether a runner should steal or not. What originally started as a plan to increase steal success became a fan-facing tool supported by machine learning. Future work could improve the model by adding ball-position data, throw direction, pitcher delivery time, catcher pop time, and steal attempts at third base or home plate.



Red Light Green Light Should He Go





Acknowledgements

I would first like to thank SMT for providing the code and allowing us to participate in this Data challenge. I would also like to thank Dr. Meredith Wills, Billy Fryer, Cameron Adams, and Andrew Steenkamer for feedback during office hours, coding help and support.

Citations

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., et al. “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, 2011. Used as the main Python machine learning library reference.

Scikit-learn Developers. *Scikit-learn Documentation*. Used for the definitions and coding help

SMT Data Challenge. 2026. *Provided Baseball Tracking Data and Data Challenge Tutorials*.

TeamRankings. 2026. *MLB Stolen Base Attempts per Game and Team Steal Attempt Data*.

Wills, Meredith. Office hours

Adams, Cameron. Office hours

Fryer, Billy. Office hours and SMT Data Challenge tutorial

Steenkamer, Andrew. Office hours